

AllianceOS Phase 1 — Complete Developer Guide

All code is copy-paste ready. Dev time: 5 days, not 3 weeks. This version includes the AI Knowledge Base — the brain that makes OpenAI output ABM-specific recommendations instead of generic scores.

WHY KNOWLEDGE BASE IS NOT OPTIONAL | WITHOUT IT AI IS BLIND

Without Knowledge Base	With Knowledge Base
OpenAI scores generically. No ABM-specific offer.	OpenAI knows your courses, fees, MoU models, and districts.
Output: score=72, offer=Training Program	Output: score=91, course=Full Stack Tier 2 PAP model, hook=Only 12% CSE
Outreach is generic copy-paste.	Every message personalised with real hook from analysis.
Placement officer ignores message.	Placement officer replies because message is specific to their college.

1. What Goes Into The Knowledge Base

All these PDFs are already built. Dev uploads them to the KB on Day 4. OpenAI reads them before scoring every lead.

Document	Brand	Category	Already Built?
BMAcademy_Profile_v4.pdf	BM Academy	Profile + Courses	■ Yes
CoreTalents_Profile_v4.pdf	Core Talents	Company Tier Model	■ Yes
BMTechX_Internship_v4.pdf	BM TechX	Internship Program	■ Yes
College_Partnership_MoU.pdf	All Brands	MoU Terms	■ Yes
Company_Partnership_MoU.pdf	Core Talents	MoU + Fee Tiers	■ Yes
ABMGroups_MasterProfile.pdf	ABM Groups	Full Overview	■ Yes
WA_Templates_MetaSubmission.pdf	All	Outreach Templates	■ Yes
FAQs_Objections.txt	All	Objection Handling	Write — 30 mins
District_Strategy.txt	All	Target Districts TN	Write — 15 mins

■ *FAQs_Objections.txt and District_Strategy.txt — plain text files. Kamar or Karthika writes these. 45 minutes total. Copy from objection scripts in the Phase 1 Execution Guide.*

2. Tech Stack

Layer	Technology	Purpose	Version
Frontend	NextJS 14	All UI screens	14.x
Backend	NodeJS + Express	REST API	20.x LTS
Database	PostgreSQL	All data	16.x
Automation	n8n self-hosted	Workflow engine	Latest
AI	OpenAI API	Lead analysis	GPT-4o
KB Extraction	pdf-parse	PDF → text	Latest
WhatsApp	Meta Cloud API	Message sending	Existing
Email	Nodemailer	Email outreach	Latest
Styling	Tailwind CSS	UI	3.x
Alerts	Telegram Bot API	Founder alerts	—

3. Folder Structure — Complete

Folder Structure

```
allianceos/
├── frontend/
│   ├── app/
│   │   ├── dashboard/page.jsx    ← Stats overview
│   │   ├── leads/page.jsx       ← Lead table with scores
│   │   ├── upload/page.jsx      ← CSV upload
│   │   ├── pipeline/page.jsx    ← Kanban board
│   │   ├── knowledge-base/page.jsx ← KB upload + list
│   │   ├── prompts/page.jsx     ← Edit AI prompts
│   │   └── components/
│   │       ├── LeadTable.jsx
│   │       ├── PipelineKanban.jsx
│   │       ├── UploadCSV.jsx
│   │       ├── KBUpload.jsx      ← NEW
│   │       └── StatsBar.jsx
│   └── backend/
│       ├── routes/
│       │   ├── leads.js
│       │   ├── upload.js
│       │   ├── analyze.js
│       │   ├── pipeline.js
│       │   ├── knowledge.js      ← NEW — KB upload + query
│       │   └── prompts.js       ← NEW — manage AI prompts
│       ├── services/
│       │   ├── openai.js        ← Updated with KB injection
│       │   ├── scraper.js
│       │   ├── whatsapp.js
│       │   └── email.js
│       ├── db/
│       │   ├── schema.sql
│       │   └── connection.js
│       └── index.js
│   └── n8n-workflows/
│       ├── flow1_lead_import.json
│       ├── flow2_ai_analyzer.json
│       └── flow3_reply_detector.json
│   └── knowledge-docs/          ← Drop PDFs here before upload
│       ├── BMAcademy_Profile_v4.pdf
│       ├── CoreTalents_Profile_v4.pdf
│       ├── BMTechX_Internship_v4.pdf
│       ├── College_Partnership_MoU.pdf
│       ├── Company_Partnership_MoU.pdf
│       └── FAQs_Objections.txt
```

4. PostgreSQL Schema — Run This First

schema.sql

```
-- Run: psql allianceos < schema.sql
CREATE TABLE organisations (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(255) NOT NULL,
  type        VARCHAR(50)  NOT NULL, -- college | company | clinic
  website     VARCHAR(500),
  email       VARCHAR(255),
  phone       VARCHAR(20),
  location    VARCHAR(255),
  district    VARCHAR(100),
  contact_name VARCHAR(255),
  contact_role VARCHAR(100),
  industry    VARCHAR(100),
  student_count INTEGER,
  lead_source  VARCHAR(100),
  status       VARCHAR(50) DEFAULT 'new',
  created_at   TIMESTAMP DEFAULT NOW(),
  updated_at   TIMESTAMP DEFAULT NOW()
);

CREATE TABLE ai_analysis (
  id          SERIAL PRIMARY KEY,
  org_id       INTEGER REFERENCES organisations(id),
  score        INTEGER,
  offer_recommended TEXT,
  reason       TEXT,
  bm_course_match TEXT,      -- which BM Academy course fits
  core_talents_offer TEXT,   -- what CT should offer
  personalisation_hook TEXT,
  hiring_potential VARCHAR(20),
  training_potential VARCHAR(20),
  raw_json     JSONB,
  created_at   TIMESTAMP DEFAULT NOW()
);

CREATE TABLE knowledge_base (
  id          SERIAL PRIMARY KEY,
  title       VARCHAR(255),
  category    VARCHAR(100), -- profile|mou|faq|strategy|template
  brand       VARCHAR(100), -- BM Academy|Core Talents|BM TechX|All
  content     TEXT,        -- extracted text from PDF or plain text
  file_name   VARCHAR(255),
  char_count  INTEGER,
  created_at  TIMESTAMP DEFAULT NOW()
);

CREATE TABLE prompt_templates (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(100) UNIQUE, -- college_analyzer|company_analyzer etc.
  purpose     VARCHAR(255),
  prompt_text TEXT,
```

```

    active          BOOLEAN DEFAULT TRUE,
    updated_at      TIMESTAMP DEFAULT NOW()
);
CREATE TABLE outreach (
    id              SERIAL PRIMARY KEY,
    org_id          INTEGER REFERENCES organisations(id),
    channel          VARCHAR(20),    -- whatsapp | email
    msg_type         VARCHAR(10),    -- M1|M2|M3|M4
    content          TEXT,
    sent_at         TIMESTAMP,
    delivered        BOOLEAN DEFAULT FALSE,
    opened          BOOLEAN DEFAULT FALSE,
    replied          BOOLEAN DEFAULT FALSE,
    reply_text       TEXT,
    replied_at       TIMESTAMP
);
CREATE TABLE pipeline_stages (
    id              SERIAL PRIMARY KEY,
    org_id          INTEGER REFERENCES organisations(id) UNIQUE,
    stage           VARCHAR(50) DEFAULT 'new',
    assigned_to     VARCHAR(100),
    notes           TEXT,
    updated_at      TIMESTAMP DEFAULT NOW()
);
-- Insert default AI prompts
INSERT INTO prompt_templates (name, purpose, prompt_text) VALUES
('college_analyzer',
 'Analyze a college and score for Core Talents partnership',
 'You are an AI analyst for ABM Groups – BM Academy, Core Talents, and BM TechX in Pondicherry, Tamil Nadu.
Analyze the following college using the scraped website data and our offerings context below.
Return ONLY valid JSON. No markdown. No explanation outside the JSON.
Organisation: {{org_name}}
Location: {{district}}
Website Content: {{website_text}}
Our Offerings:
{{kb_context}}
Return this exact JSON structure:
{
  "score": <integer 0-100>,
  "offer_recommended": "<specific MoU type>",
  "reason": "<2 sentences why this score>",
  "bm_course_match": "<which BM Academy courses fit their students>",
  "core_talents_offer": "<what Core Talents can specifically offer>",
  "training_potential": "high|medium|low",
  "placement_potential": "high|medium|low",
  "personalisation_hook": "<one specific sentence about this college that makes outreach feel personal>"
}'
),
('company_analyzer',
 'Analyze a company and score for Core Talents hiring partnership',
 'You are an AI analyst for Core Talents – the talent placement division of ABM Groups, Pondicherry.
Analyze the following company using website data and our talent offerings below.
Return ONLY valid JSON. No markdown. No explanation.
Organisation: {{org_name}}
Industry: {{industry}}
Location: {{district}}
Website Content: {{website_text}}
Our Talent Offerings:
{{kb_context}}
Return this exact JSON:
{
  "score": <integer 0-100>,
  "offer_recommended": "<Free Tier | Growth Tier | Partner Retainer>",
  "reason": "<2 sentences why>",
  "skills_match": "<which of our trained skills fit this company>",
  "hiring_potential": "high|medium|low",
  "personalisation_hook": "<one specific sentence about this company>"
}'
);

```

5. Knowledge Base Route — backend/routes/knowledge.js

■ Run first: *npm install pdf-parse multer*

knowledge.js

```
// backend/routes/knowledge.js
const express = require('express');
const multer = require('multer');
const pdfParse = require('pdf-parse');
const fs = require('fs');
const db = require('../db/connection');
const router = express.Router();
const upload = multer({ dest: 'uploads/kb/' });

// POST /api/knowledge/upload
// Accepts: PDF, DOCX, or TXT file
router.post('/upload', upload.single('file'), async (req, res) => {
  const { title, category, brand } = req.body;
  try {
    let text = '';
    if (req.file.mimetype === 'application/pdf') {
      const buffer = fs.readFileSync(req.file.path);
      const data = await pdfParse(buffer);
      // Clean extracted text
      text = data.text
        .replace(/\n{3,}/g, '\n\n') // collapse excessive newlines
        .replace(/\s{2,}/g, ' ') // collapse spaces
        .trim()
        .slice(0, 10000); // limit to 10k chars per doc
    } else {
      // Plain text or other
      text = fs.readFileSync(req.file.path, 'utf8')
        .slice(0, 10000);
    }
    fs.unlinkSync(req.file.path); // clean up temp file
    if (!text || text.length < 50) {
      return res.status(400).json({
        success: false,
        message: 'Could not extract text from file. Check PDF is not scanned image.'
      });
    }
    const { rows } = await db.query(
      `INSERT INTO knowledge_base
      (title, category, brand, content, file_name, char_count)
      VALUES ($1,$2,$3,$4,$5,$6)
      RETURNING id`,
      [title, category, brand,
        text, req.file.originalname, text.length]
    );
    res.json({
      success: true,
```

```

      id: rows[0].id,
      chars_extracted: text.length,
      message: `Knowledge base updated: ${title}`
    });
  } catch (err) {
    console.error('KB upload error:', err);
    res.status(500).json({ success: false, message: err.message });
  }
});
// GET /api/knowledge - list all KB entries
router.get('/', async (req, res) => {
  const { rows } = await db.query(`
    SELECT id, title, category, brand,
           char_count, created_at,
           LEFT(content, 200) AS preview
    FROM knowledge_base
    ORDER BY created_at DESC
  `);
  res.json({ success: true, documents: rows });
});
// DELETE /api/knowledge/:id
router.delete('/:id', async (req, res) => {
  await db.query('DELETE FROM knowledge_base WHERE id=$1', [req.params.id]);
  res.json({ success: true });
});
// GET /api/knowledge/context?type=college
// Used internally by AI analyzer
router.get('/context', async (req, res) => {
  const { type } = req.query;
  const context = await getKBContext(type);
  res.json({ success: true, context, length: context.length });
});
module.exports = router;

```

6. AI Analyzer — Updated With KB Injection

■ Run first: *npm install openai axios* — replace old *openai.js* with this

openai.js — Full AI Service with KB

[illegible]

[illegible]


```

        personalisation_hook: '',
        training_potential: 'medium',
        placement_potential: 'medium',
        hiring_potential: 'medium',
        bm_course_match: '',
        core_talents_offer: ''
    });
}
// Save analysis to database
await db.query(`
    INSERT INTO ai_analysis
    (org_id, score, offer_recommended, reason,
     bm_course_match, core_talents_offer,
     personalisation_hook, hiring_potential,
     training_potential, raw_json)
    VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10)
    ON CONFLICT DO NOTHING
`, [
    org.id,
    result.score || 50,
    result.offer_recommended || '',
    result.reason || '',
    result.bm_course_match || result.skills_match || '',
    result.core_talents_offer || '',
    result.personalisation_hook || '',
    result.hiring_potential || 'medium',
    result.training_potential || 'medium',
    result
]);
// Update org status
await db.query(
    `UPDATE organisations
     SET status='analysed', updated_at=NOW()
     WHERE id=$1`,
    [org.id]
);
// Telegram alert for HOT leads (score >= 85)
if ((result.score || 0) >= 85) {
    const msg =
        `■ HOT LEAD\n` +
        `Org: ${org.name}\n` +
        `Score: ${result.score}/100\n` +
        `Offer: ${result.offer_recommended}\n` +
        `Hook: ${result.personalisation_hook}\n` +
        `District: ${org.district}`;
    await sendTelegramAlert(msg);
}
return result;

```

[illegible]

7. CSV Upload Route — backend/routes/upload.js

upload.js

```
// backend/routes/upload.js
const express = require('express');
const multer = require('multer');
const csv = require('csv-parser');
const fs = require('fs');
const db = require('../db/connection');
const { analyzeOrganisation } = require('../services/openai');
const router = express.Router();
const upload = multer({ dest: 'uploads/leads/' });
// Column name maps – handles different sheet formats
const COL = {
  name: ['College Name', 'Company Name', 'Clinic Name', 'Name'],
  email: ['Email', 'PO Email', 'HR Email'],
  phone: ['Phone', 'PO Phone', 'HR Phone', 'Phone Number'],
  website: ['Website', 'URL'],
  district: ['District'],
  contact: ['Placement Officer', 'HR Name', 'HR Manager', 'Contact Person', 'Doctor Name'],
  role: ['Designation', 'Role'],
  industry: ['Industry', 'Type', 'Specialty'],
  students: ['Student Strength', 'Students'],
};
function getCol(row, keys) {
  for (const k of keys) {
    if (row[k] !== undefined && row[k] !== '') return row[k];
  }
  return '';
}
// POST /api/upload/csv
router.post('/csv', upload.single('file'), async (req, res) => {
  const { type } = req.body; // college | company | clinic
  const rows_data = [];
  fs.createReadStream(req.file.path)
    .pipe(csv())
    .on('data', row => rows_data.push(row))
    .on('end', async () => {
      fs.unlinkSync(req.file.path);
      let imported = 0, skipped = 0, errors = 0;
      const newIds = [];
      for (const row of rows_data) {
        const name = getCol(row, COL.name);
        if (!name) { skipped++; continue; }
        try {
          const { rows } = await db.query(`
            INSERT INTO organisations
              (name,type,email,phone,website,location,`
```

```

        district,contact_name,contact_role,
        industry,student_count,lead_source)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12)
ON CONFLICT DO NOTHING
RETURNING id
`, [
  name, type,
  getCol(row, COL.email),
  getCol(row, COL.phone),
  getCol(row, COL.website),
  getCol(row, COL.district),
  getCol(row, COL.district),
  getCol(row, COL.contact),
  getCol(row, COL.role),
  getCol(row, COL.industry),
  parseInt(getCol(row, COL.students)) || null,
  'CSV Upload'
]);
if (rows.length > 0) {
  imported++;
  newIds.push(rows[0].id);
} else {
  skipped++; // duplicate
}
} catch (err) {
  errors++;
  console.error('Row error:', name, err.message);
}
}
// Trigger AI analysis in background – don't await
if (newIds.length > 0) {
  setTimeout(async () => {
    for (const id of newIds) {
      try {
        const { rows } = await db.query(
          'SELECT * FROM organisations WHERE id=$1', [id]
        );
        if (rows[0]) await analyzeOrganisation(rows[0]);
        await new Promise(r => setTimeout(r, 1200)); // rate limit
      } catch (e) { console.error('Analysis error:', e.message); }
    }
  }, 100);
}
res.json({
  success: true, imported, skipped, errors,
  message: `${imported} imported. Analysis running. Check leads in ~${Math.ceil(newIds.length * 1.5 / 60)} mi
ns.`
});
})
.on('error', err => {

  res.status(500).json({ success: false, message: err.message });
});
});
module.exports = router;

```

8. Pipeline Route — backend/routes/pipeline.js

pipeline.js

```
// backend/routes/pipeline.js
const express = require('express');
const db      = require('../db/connection');
const router  = express.Router();
// GET /api/pipeline?type=college
router.get('/', async (req, res) => {
  const { type } = req.query;
  const typeFilter = type ? 'AND o.type = $1' : '';
  const params     = type ? [type] : [];
  const { rows } = await db.query(`
    SELECT
      o.id, o.name, o.type, o.email, o.phone,
      o.district, o.status, o.contact_name,
      a.score, a.offer_recommended,
      a.personalisation_hook, a.bm_course_match,
      a.core_talents_offer,
      COALESCE(p.stage, 'new')           AS stage,
      COALESCE(p.assigned_to, '')        AS assigned_to,
      COALESCE(p.notes, '')              AS notes,
      p.updated_at                       AS stage_updated
    FROM organisations o
    LEFT JOIN ai_analysis a ON a.org_id = o.id
    LEFT JOIN pipeline_stages p ON p.org_id = o.id
    ${typeFilter}
    ORDER BY COALESCE(a.score, 0) DESC
  `, params);
  // Group by stage for Kanban
  const stages = ['new', 'contacted', 'replied', 'meeting', 'mou', 'partner'];
  const pipeline = {};
  stages.forEach(st => pipeline[st] = []);
  rows.forEach(row => {
    const st = row.stage || 'new';
    if (pipeline[st]) pipeline[st].push(row);
    else pipeline['new'].push(row);
  });
  res.json({
    success: true,
    pipeline,
    totals: Object.fromEntries(
      Object.entries(pipeline).map(([k, v]) => [k, v.length])
    )
  });
});
// PATCH /api/pipeline/:id/stage
router.patch('/:id/stage', async (req, res) => {
  const { id } = req.params;
```

```

const { stage, assigned_to, notes } = req.body;
await db.query(`
  INSERT INTO pipeline_stages
    (org_id, stage, assigned_to, notes, updated_at)
  VALUES ($1,$2,$3,$4,NOW())
  ON CONFLICT (org_id) DO UPDATE
  SET stage=$2, assigned_to=$3,
      notes=COALESCE($4, pipeline_stages.notes),
      updated_at=NOW()
`, [id, stage, assigned_to || '', notes || '']);
await db.query(
  `UPDATE organisations SET status=$1, updated_at=NOW() WHERE id=$2`,
  [stage, id]
);
res.json({ success: true, moved_to: stage });
});
// GET /api/pipeline/hot - score >= 85, not yet contacted
router.get('/hot', async (req, res) => {
  const { rows } = await db.query(`
    SELECT o.name, o.district, o.phone, o.email,
      a.score, a.offer_recommended,
      a.personalisation_hook, a.bm_course_match
    FROM organisations o
    JOIN ai_analysis a ON a.org_id = o.id
    LEFT JOIN pipeline_stages p ON p.org_id = o.id
    WHERE a.score >= 85
    AND COALESCE(p.stage,'new') IN ('new','contacted')
    ORDER BY a.score DESC
    LIMIT 20
  `);
  res.json({ success: true, hot_leads: rows, count: rows.length });
});
module.exports = router;

```

9. Server Entry — backend/index.js

index.js

```

// backend/index.js
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const uploadRouter = require('./routes/upload');
const leadsRouter = require('./routes/leads');
const pipelineRouter = require('./routes/pipeline');
const knowledgeRouter = require('./routes/knowledge');
const analyzeRouter = require('./routes/analyze');
const app = express();
app.use(cors());
app.use(express.json());
// Routes
app.use('/api/upload', uploadRouter);
app.use('/api/leads', leadsRouter);
app.use('/api/pipeline', pipelineRouter);
app.use('/api/knowledge', knowledgeRouter);
app.use('/api/analyze', analyzeRouter);
// Health check
app.get('/health', (req, res) => {
  res.json({ status:'ok', time: new Date().toISOString() });
});
const PORT = process.env.PORT || 3001;
app.listen(PORT, () => {
  console.log(`AllianceOS API running on port ${PORT}`);
  console.log(`Environment: ${process.env.NODE_ENV}`);
});

```

10. Frontend — KB Upload Screen

knowledge-base/page.jsx

```
// frontend/app/knowledge-base/page.jsx
'use client';
import { useState, useEffect } from 'react';
export default function KnowledgeBase() {
  const [docs, setDocs] = useState([]);
  const [uploading, setUploading] = useState(false);
  const [form, setForm] = useState({
    title: '', category: 'profile', brand: 'BM Academy'
  });
  useEffect(() => { fetchDocs(); }, []);
  const fetchDocs = () =>
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/knowledge`)
      .then(r => r.json())
      .then(d => setDocs(d.documents || []));
  const handleUpload = async (e) => {
    e.preventDefault();
    const file = e.target.file.files[0];
    if (!file) return alert('Select a file');
    setUploading(true);
    const fd = new FormData();
    fd.append('file', file);
    fd.append('title', form.title || file.name);
    fd.append('category', form.category);
    fd.append('brand', form.brand);
    const res = await fetch(
      `${process.env.NEXT_PUBLIC_API_URL}/knowledge/upload`,
      { method: 'POST', body: fd }
    );
    const data = await res.json();
    setUploading(false);
    if (data.success) { fetchDocs(); alert(`Uploaded: ${data.chars_extracted} chars extracted`); }
    else alert('Upload failed: ' + data.message);
  };
  const deleteDoc = async (id) => {
    if (!confirm('Delete this KB document?')) return;
    await fetch(`${process.env.NEXT_PUBLIC_API_URL}/knowledge/${id}`,
      { method: 'DELETE' });
    fetchDocs();
  };
  return (
    <div className="max-w-4xl mx-auto p-6">
      <h1 className="text-2xl font-bold mb-6">AI Knowledge Base</h1>
      <p className="text-sm text-gray-500 mb-6">
        Upload ABM Groups profile PDFs here. The AI reads these before
      </p>
    </div>
  );
}
```

```

    scoring every lead. Without documents, scores are generic.
  </p>
  { /* Upload Form */ }
  <form onSubmit={handleUpload}
    className="bg-white border rounded-lg p-6 mb-8 shadow-sm">
    <h2 className="font-semibold mb-4">Upload Document</h2>
    <div className="grid grid-cols-3 gap-3 mb-3">
      <input name="title" placeholder="Document title"
        className="border rounded px-3 py-2 text-sm"
        value={form.title}
        onChange={e => setForm({...form, title: e.target.value})} />
      <select className="border rounded px-3 py-2 text-sm"
        value={form.category}
        onChange={e => setForm({...form, category: e.target.value})}>
        <option value="profile">Profile</option>
        <option value="mou">MoU / Agreement</option>
        <option value="faq">FAQ / Objections</option>
        <option value="strategy">Strategy / Targets</option>
        <option value="template">Templates</option>
      </select>
      <select className="border rounded px-3 py-2 text-sm"
        value={form.brand}
        onChange={e => setForm({...form, brand: e.target.value})}>
        <option>BM Academy</option>
        <option>Core Talents</option>
        <option>BM TechX</option>
        <option>All</option>
      </select>
    </div>
    <input type="file" name="file" accept=".pdf,.txt"
      className="w-full border rounded px-3 py-2 mb-3 text-sm" />
    <button type="submit"
      className="bg-slate-800 text-white px-6 py-2 rounded font-medium
        hover:bg-slate-700 disabled:opacity-50"
      disabled={uploading}>
      {uploading ? 'Uploading...' : 'Upload to Knowledge Base'}
    </button>
  </form>
  { /* Existing Documents */ }
  <table className="w-full text-sm border-collapse">
    <thead>
      <tr className="bg-slate-800 text-white">
        <th className="p-2 text-left">Title</th>
        <th className="p-2 text-center">Brand</th>
        <th className="p-2 text-center">Category</th>
        <th className="p-2 text-center">Chars</th>
        <th className="p-2 text-center">Action</th>
      </tr>
    </thead>
    <tbody>

```



```

{docs.map((doc, i) => (
  <tr key={doc.id} className={i%2===0?'bg-white':'bg-gray-50'}>
    <td className="p-2">{doc.title}</td>
    <td className="p-2 text-center">{doc.brand}</td>
    <td className="p-2 text-center">{doc.category}</td>
    <td className="p-2 text-center text-gray-500">
      {doc.char_count?.toLocaleString()}
    </td>
    <td className="p-2 text-center">
      <button onClick={() => deleteDoc(doc.id)}
        className="text-red-500 hover:text-red-700 text-xs">
          Remove
        </button>
      </td>
    </tr>
  )
)}}
{docs.length === 0 &&
  <tr><td colspan={5} className="p-4 text-center text-gray-400">
    No documents yet. Upload ABM Groups PDFs to activate AI brain.
  </td></tr>
}
</tbody>
</table>
</div>
);
}

```

[illegible]

Terminal — All Install Commands

[illegible]

13. 5-Day Build Plan — Code Is Already Written

Code is copy-paste. Dev reads, configures, tests. Kamar reviews end of each day via Telegram.

DAY 1 Foundation — Copy + Configure

- ✓ Create PostgreSQL database: createdb allianceos
- ✓ Run schema.sql — all 6 tables created
- ✓ Create backend/ folder, copy all route files
- ✓ Copy openai.js, upload.js, pipeline.js, knowledge.js
- ✓ Copy index.js — register all routes
- ✓ Fill .env with real keys (OpenAI, Telegram, Gmail, Meta)
- ✓ npm install all backend packages
- ✓ Test: node index.js — server starts on port 3001
- ✓ Test: GET /health — returns {status:ok}

Result: Server running. DB connected. All routes responding.

DAY 2 CSV Upload + AI Analysis Working

- ✓ Test CSV upload with 5 real college rows
- ✓ Confirm rows appear in PostgreSQL organisations table
- ✓ Confirm AI analyzer triggers in background
- ✓ Check ai_analysis table — rows inserted with scores
- ✓ Test Telegram alert — upload a row with score 85+
- ✓ Confirm Telegram message arrives on Kamar's phone
- ✓ Test Knowledge Base upload — upload BMAcademy_Profile_v4.pdf
- ✓ Confirm text extraction: char_count > 1000
- ✓ Re-run analyzer — confirm KB context appears in prompt

Result: CSV upload working. AI scoring with KB context. Telegram alerts live.

DAY 3 Frontend — All Screens

- ✓ npx create-next-app in frontend/ folder
- ✓ Copy LeadTable.jsx, UploadCSV.jsx components
- ✓ Copy knowledge-base/page.jsx — KB upload screen
- ✓ Copy upload/page.jsx — CSV upload screen
- ✓ Build leads/page.jsx — wire to GET /api/leads
- ✓ Build pipeline/page.jsx — Kanban with stage drag
- ✓ Build dashboard/page.jsx — stats from pipeline totals
- ✓ Test full flow: upload CSV in browser → see scores in table
- ✓ Test KB upload in browser → see doc in list

Result: Frontend working. Full flow visible in browser.

DAY 4 n8n + Reply Detection

- ✓ npm install n8n -g && n8n start
- ✓ Create Webhook trigger node — POST /allianceos-new-lead
- ✓ Create HTTP Request node — calls /api/analyze/batch
- ✓ Create IF node — score >= 85 branches to Telegram
- ✓ Create Gmail trigger node — polls inbox every 15 mins
- ✓ Create Code node — check sender against organisations table
- ✓ Create HTTP node — PATCH /api/pipeline/:id/stage → replied
- ✓ Create Telegram node — alert Satish when reply detected
- ✓ Test full n8n flow with 2 real leads

Result: Full automation running. Replies auto-detected. Satish gets Telegram alerts.

DAY 5 Full Test + Deploy

- ✓ Upload 20 real leads: 10 colleges, 5 companies, 5 clinics
- ✓ Upload all 7 KB documents (PDFs already built)
- ✓ Run AI analysis on all 20 — check scores and hooks
- ✓ Review 3 highest scoring leads — confirm personalisation is specific
- ✓ Fix any prompt issues in prompts/page.jsx
- ✓ Move 5 leads through pipeline stages manually — test Kanban
- ✓ Deploy backend: pm2 start index.js --name allianceos-api
- ✓ Deploy frontend: npm run build && npm start
- ✓ Deploy n8n: pm2 start n8n --name n8n
- ✓ Share AllianceOS URL with Kamar — review together

Result: AllianceOS Phase 1 is LIVE. AI brain active. Ready for outreach.

Dev starts Day 1 today. Kamar reviews progress at 7PM each evening via Telegram. Day 5 = AllianceOS Phase 1 live. AI brain active. First leads being scored.